软件设计模式

项目考查报告

学　　院　软件工程

专　　业　数据科学

姓　　名　毛乾屹

学　　号　2022083036　班级　221

2024 年　月　日

目　录

1　项目概述 1

1.1 项目简介 1

1.2 项目建设的主要原则 1

1.3 项目目的 1

2　考查题目与要求 2

3　设计与开发 2

3.1　主要功能设计 2

3.2　类和接口设计 6

3.3　前端页面设计（如有需要） 8

3.3.1　前端界面设计效果 8

3.3.2　关键代码 10

3.4　功能模块 1 添加事务的开发与实现 16

3.5　功能模块 2 删除事务的开发与实现 17

3.6　功能模块 3 修改事务的开发与实现 18

3.7　功能模块 4 查询事务的开发与实现 19

3.8　功能模块 5 数据持久化的开发与实现 21

3.9　功能模块 6 图形用户界面的开发与实现 23

4　运行结果测试 25

5　使用设计模式总结 29

1 项目概述

说明本项目的主要功能、项目建设的主要原则、目的等内容。

1.1 项目简介

本项目旨在开发一款电子记账系统，主要功能包括记账数据的增、删、改、查（CRUD）操作。系统采用多种软件设计模式，以提高代码的可维护性、扩展性和复用性。数据持久化方面，系统选择将记账数据保存到数据库中，确保程序在重新启动后能够加载历史数据。此外，系统实现了前端用户界面（vue+Element-ui），提供友好的用户交互体验。

1.2 项目建设的主要原则

模块化设计：通过分层架构和设计模式，将系统功能模块化，便于维护和扩展。

高内聚低耦合：各模块内部功能紧密相关，模块之间依赖性低，提高系统的灵活性。

代码复用：采用设计模式实现代码的复用，减少冗余代码，提高开发效率。

数据持久化：确保记账数据能够持久保存，保证数据的完整性和可靠性。

用户友好：通过图形界面设计，提升用户操作的便捷性和体验。

1.3 项目目的

实现基本记账功能：完成记账数据的增、删、改、查等基本功能。

应用设计模式：在项目中应用至少三种设计模式，提升代码质量和系统架构。

数据持久化：实现记账数据的持久化存储，确保数据在程序重启后依然可用。

独立开发：通过独立完成项目，提升个人的软件开发能力和设计思维。

全面测试：完成系统的全面测试，确保各项功能的正确性和稳定性。

图形界面实现：通过实现图形界面，增强系统的可用性和用户体验。

2 考查题目与要求

考查题目：采用多种软件设计模式的电子记账系统。

考查要求：

(1) 需要体现至少 3 种设计模式，并说明各个模式在项目中的作用；

(2) 要完成记账数据的增、删、改、查等功能，要求数据保存到文件或数据库中，以便程序在重新启动后也能加载历史数据；

(3) 每个同学独立完成，不得抄袭；

(4) 需要完整的测试记录；

(5) 可以不用设计图形界面，但如果实现了图形界面且功能实现正确，加获得 10 分加分。

3 设计与开发

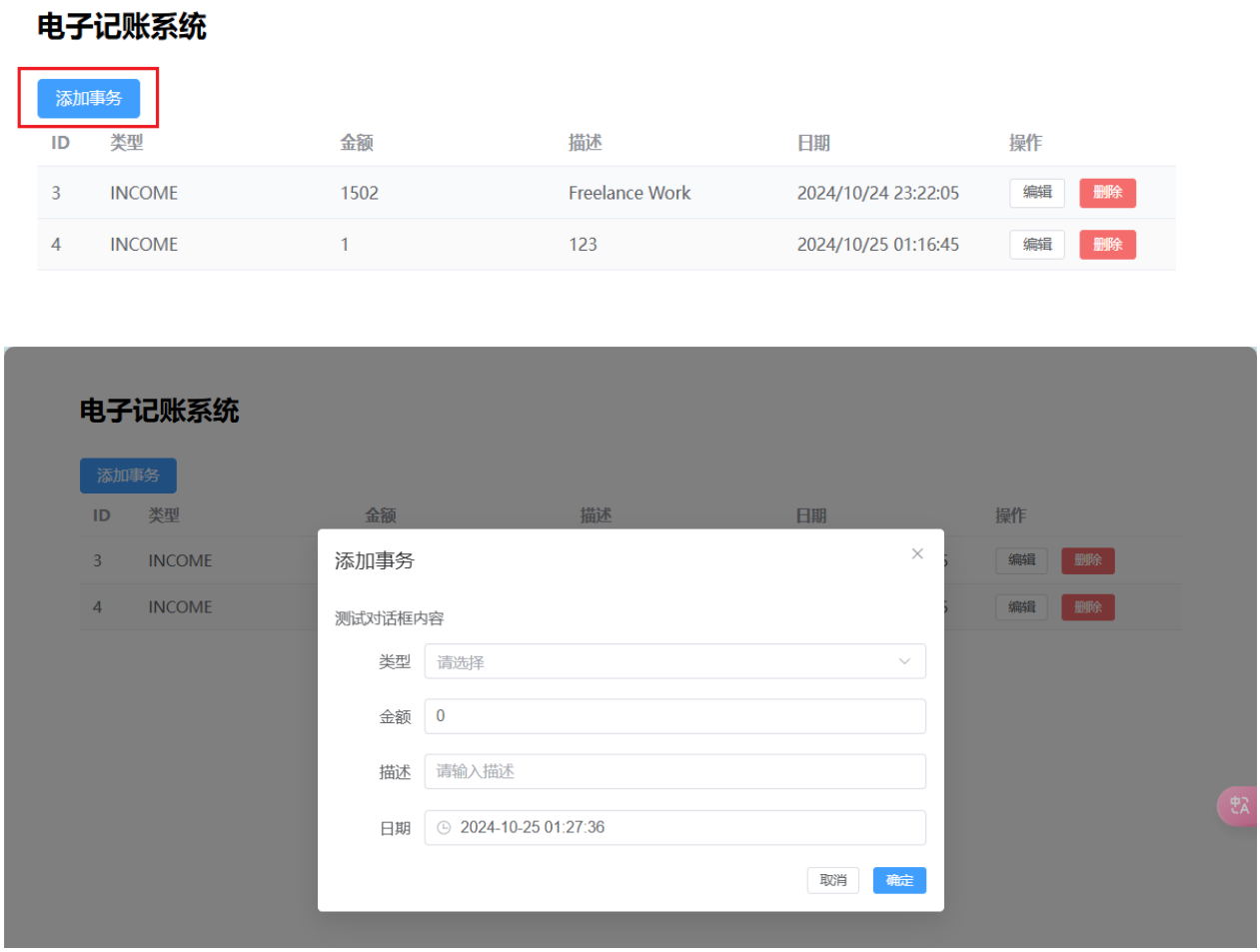
本部分请使用截图并配以文字的形式说明针对实验题目的每项要求的完成过程和结果。

3.1 主要功能设计

说明本项目的主要功能，可以使用用例图、表等辅助说明各个功能。功能设计除了包含题目的基本要求以外，也可根据自己的理解，创新开发一些新功能（有加分）。

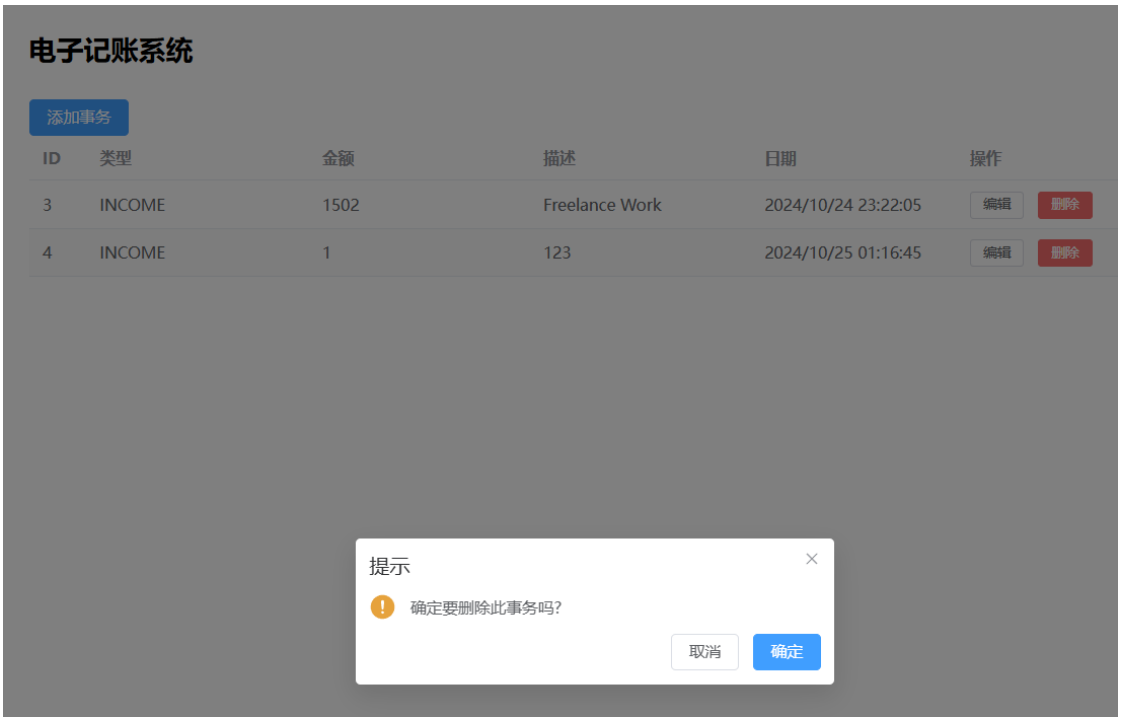
本项目的主要功能包括：

1. **添加事务**：用户可以添加新的记账记录，包括类型（收入或支出）、金额、描述和日期。



2. **删除事务**：用户可以删除不需要的记账记录。





3. **修改事务**：用户可以编辑已有的记账记录。





4. **查询事务**：用户可以查看所有记账记录，支持按日期、类型等条件筛选。



5. **数据持久化**

系统将记账数据保存到数据库中，确保数据在程序重启后依然可用

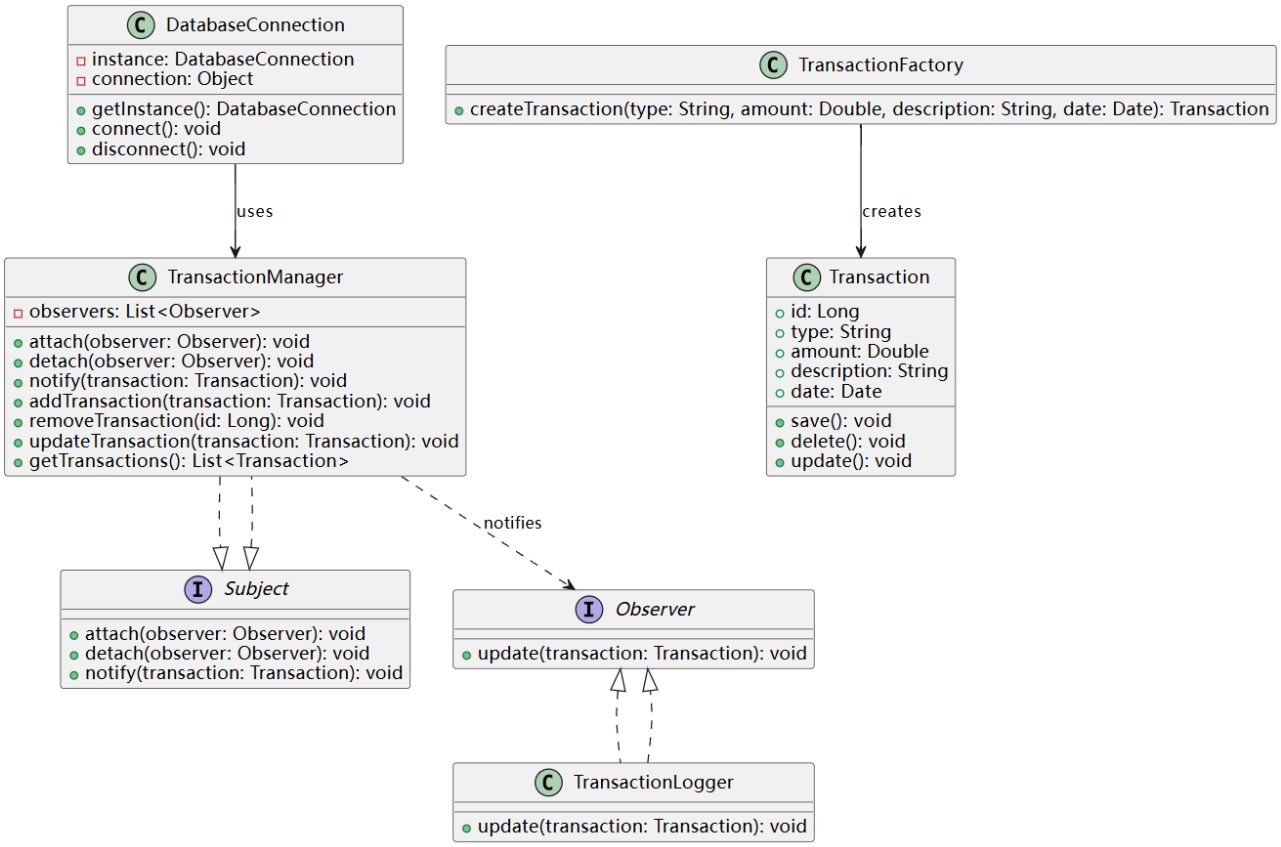

```
[
    {
        "id": 3,
        "type": "INCOME",
        "amount": 1502,
        "description": "Freelance Work",
        "date": "2024-10-24T15:22:05.964+00:00"
    },
    {
        "id": 4,
        "type": "INCOME",
        "amount": 1,
        "description": "123",
        "date": "2024-10-24T17:16:45.962+00:00"
    }
]
```


6 **图形用户界面**提供友好的图形界面，方便用户进行操作和查看数据。



3.2 类和接口设计

以文字或 UML 类图设计本实验需要的类和接口



类与接口说明

1. DatabaseConnection（单例模式）
   - **作用：**负责与数据库建立连接，确保整个应用程序中只有一个数据库连接实例。
   - **主要方法：**
     - `getInstance()`:获取单例实例。
     - `connect()`:建立数据库连接。
     - `disconnect()`:断开数据库连接。
2. TransactionFactory（工厂模式）
   - **作用：**负责创建不同类型的记账事务对象，简化对象的创建过程。
   - **主要方法：**
     - `createTransaction(type,amount,description,date)`:根据类型创建事务对象。
3. Observer（观察者模式）

- **作用**：定义观察者接口，允许不同的观察者在事务发生变化时接收通知。
- **主要方法**：
  - update(transaction):接收事务更新通知。

4. **TransactionLogger（具体观察者）**
   - **作用**：实现 Observer 接口，负责记录事务的变化日志。
   - **主要方法**：
     - update(transaction):记录事务变化。
5. **Subject（观察者模式）**
   - **作用**：定义主题接口，允许观察者注册和注销，并在事务变化时通知观察者。
   - **主要方法**：
     - attach(observer):注册观察者。
     - detach(observer):注销观察者。
     - notify(transaction):通知所有注册的观察者事务的变化。
6. **TransactionManager（具体主题）**
   - **作用**：管理记账事务的增、删、改、查操作，并在事务变化时通知观察者。
   - **主要方法**：
     - attach(observer):注册观察者。
     - detach(observer):注销观察者。
     - notify(transaction):通知观察者事务变化。
     - addTransaction(transaction):添加新事务。
     - removeTransaction(id):删除指定事务。
     - updateTransaction(transaction):更新指定事务。
     - getTransactions():获取所有事务。
7. **Transaction（实体类）**
   - **作用**：表示记账事务的实体，包含事务的详细信息。
   - **主要属性**：
     - id:事务唯一标识。
     - type:事务类型（收入或支出）。
     - amount:事务金额。
     - description:事务描述。
     - date:事务日期。
   - **主要方法**：
     - save():保存事务到数据库。
     - delete():从数据库中删除事务。
     - update():更新事务信息。

3.3 前端页面设计（如有需要）

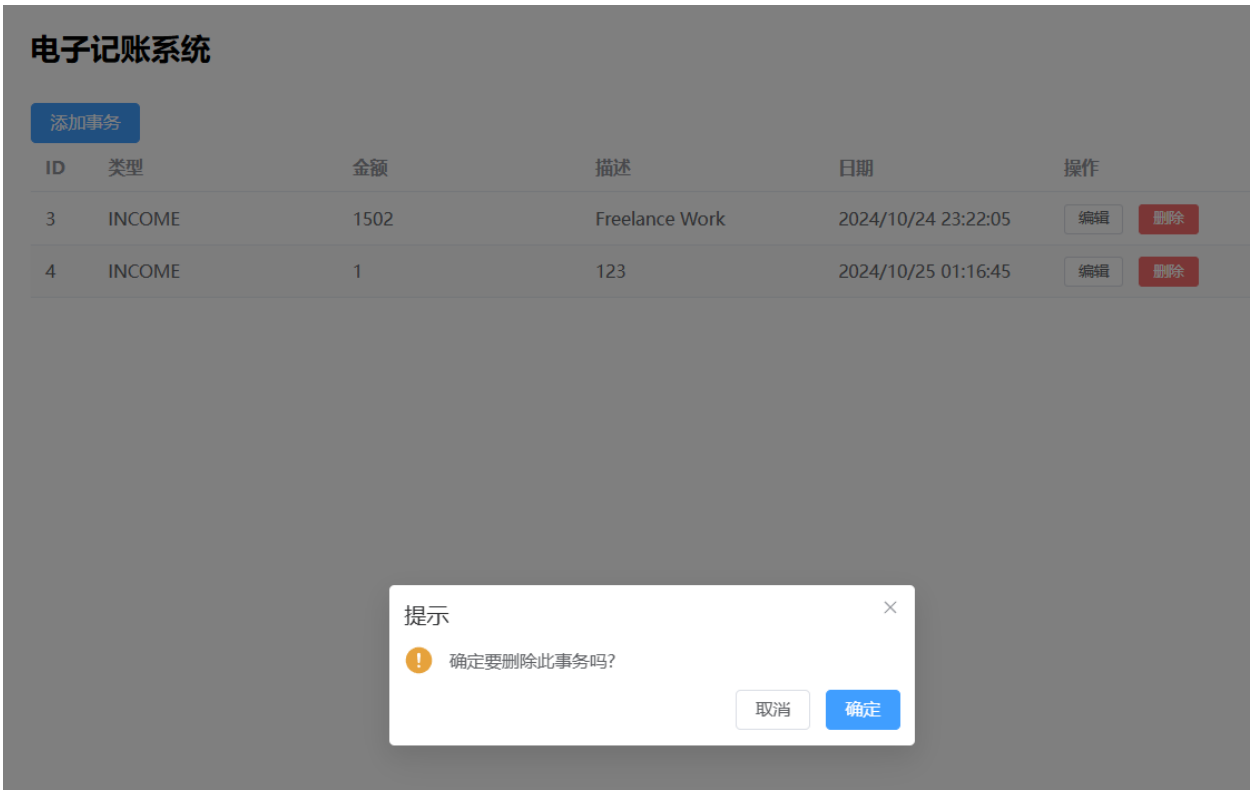
前端界面设计效果以及关键代码

3.3.1 前端界面设计效果






电子记账系统
添加事务
ID
类型
金额
描述
日期
操作
3
INCOME
4
INCOME
编辑
删除
编辑事务
测试对话框内容
类型
请选择
金额
0
描述
请输入描述
日期
2024-10-25 01:27:36
取消
确定


电子记账系统
添加事务
ID
类型
金额
描述
日期
操作
3
INCOME
1502
Freelance Work
2024/10/24 23:22:05
4
INCOME
1
123
2024/10/25 01:16:45
编辑
删除
提示
确定要删除此事务吗?
取消
确定

3.3.2 关键代码

TransactionForm.vue

```
<!-- src/components/TransactionForm.vue -->
<template>
  <div>
    <el-form :model="form" label-width="80px">
      <el-form-item label="类型">
        <el-select v-model="form.type" placeholder="请选择">
          <el-option label="收入" value="INCOME"></el-option>
          <el-option label="支出" value="EXPENSE"></el-option>
        </el-select>
      </el-form-item>
      <el-form-item label="金额">
        <el-input v-model.number="form.amount" type="number"
placeholder="请输入金额"></el-input>
      </el-form-item>
      <el-form-item label="描述">
        <el-input v-model="form.description" placeholder="请输入描述
"></el-input>
      </el-form-item>
      <el-form-item label="日期">
        <el-date-picker
            v-model="form.date"
            type="datetime"
            placeholder="选择日期和时间"
            style="width: 100%;">
        </el-date-picker>
      </el-form-item>
    </el-form>
    <div class="dialog-footer">
      <el-button size="small" @click="$emit('close')">取消</el-button>
      <el-button size="small" type="primary" @click="submitForm">确定
</el-button>
    </div>
  </div>
</template>

<script>
export default {
```

```
  name: 'TransactionForm',
  props: {
    transaction: {
      type: Object,
      default: null
    }
  },
  data() {
    return {
      form: {
        id: null,
        type: '',
        amount: 0.0,
        description: '',
        date: new Date()
      }
    }
  },
  methods: {
    submitForm() {
      console.log('提交表单', this.form) // 调试用
      // 表单验证
      if (!this.form.type) {
        this.$message.error('类型不能为空')
        return
      }
      if (!this.form.amount || this.form.amount <= 0) {
        this.$message.error('金额必须大于 0')
        return
      }
      if (!this.form.date) {
        this.$message.error('日期不能为空')
        return
      }
      // 触发提交事件
      this.$emit('submit', { ...this.form })
    }
  },
  created() {
    if (this.transaction) {
      this.form = { ...this.transaction }
```

```
      this.form.date = new Date(this.transaction.date)
    }
  }
}
</script>

<style scoped>
.dialog-footer {
  text-align: right;
  margin-top: 20px;
}
</style>
```

TransactionList.vue

```
<!-- src/components/TransactionList.vue -->
<template>
  <div>
    <el-button type="primary" @click="openAddForm">添加事务</el-button>
    <el-table :data="transactions" style="width: 100%" stripe>
      <el-table-column prop="id" label="ID" width="50"></el-table-
column>
      <el-table-column prop="type" label="类型"></el-table-column>
      <el-table-column prop="amount" label="金额"></el-table-column>
      <el-table-column prop="description" label="描述"></el-table-
column>
      <el-table-column prop="date" label="日期" width="180">
        <template #default="scope">
          {{ formatDate(scope.row.date) }}
        </template>
      </el-table-column>
      <el-table-column label="操作" width="180">
        <template #default="scope">
          <el-button size="small" @click="openEditForm(scope.row)">编辑
</el-button>
          <el-button size="small" type="danger"
@click="deleteTransaction(scope.row.id)">删除</el-button>
        </template>
      </el-table-column>
    </el-table>

    <!-- 事务表单弹窗 -->
```

```
    <el-dialog :title="dialogTitle" v-model="dialogVisible">
      <p>测试对话框内容</p>
      <TransactionForm
          :transaction="currentTransaction"
          @submit="handleFormSubmit"
          @close="dialogVisible = false"
      ></TransactionForm>
    </el-dialog>
  </div>
</template>

<script>
import TransactionService from '../services/TransactionService'
import TransactionForm from './TransactionForm.vue'

export default {
  name: 'TransactionList',
  components: {
    TransactionForm
  },
  data() {
    return {
      transactions: [],
      dialogVisible: false,
      dialogTitle: '',
      currentTransaction: null
    }
  },
  methods: {
    fetchTransactions() {
      TransactionService.getAllTransactions()
          .then(response => {
            this.transactions = response.data
          })
          .catch(error => {
            console.error(error)
            this.$message.error('获取事务失败')
          })
    },
    openAddForm() {
      console.log('打开添加事务表单') // 调试用
```

```
      this.dialogTitle = '添加事务'
      this.currentTransaction = null
      this.dialogVisible = true
    },
    openEditForm(transaction) {
      console.log('打开编辑事务表单', transaction) // 调试用
      this.dialogTitle = '编辑事务'
      this.currentTransaction = { ...transaction }
      this.dialogVisible = true
    },
    handleFormSubmit(transaction) {
      if (transaction.id) {
        // 编辑事务
        TransactionService.updateTransaction(transaction.id,
transaction)
            .then(() => {
              this.$message.success('事务更新成功')
              this.fetchTransactions()
              this.dialogVisible = false
            })
            .catch(error => {
              console.error(error)
              this.$message.error('事务更新失败')
            })
      } else {
        // 添加事务
        TransactionService.createTransaction(transaction)
            .then(() => {
              this.$message.success('事务添加成功')
              this.fetchTransactions()
              this.dialogVisible = false
            })
            .catch(error => {
              console.error(error)
              this.$message.error('事务添加失败')
            })
      }
    },
    deleteTransaction(id) {
      this.$confirm('确定要删除此事务吗?', '提示', {
        confirmButtonText: '确定',
```

```
        cancelButtonText: '取消',
        type: 'warning'
      }).then(() => {
        TransactionService.deleteTransaction(id)
            .then(() => {
              this.$message.success('事务删除成功')
              this.fetchTransactions()
            })
            .catch(error => {
              console.error(error)
              this.$message.error('事务删除失败')
            })
      }).catch(() => {
        // 取消删除
      })
    },
    formatDate(dateStr) {
      const date = new Date(dateStr)
      return date.toLocaleString()
    }
  },
  created() {
    this.fetchTransactions()
  }
}
</script>

<style scoped>
</style>
```

3.4 功能模块 1 添加事务的开发与实现

以代码+文字的方式说明各个功能模块的实现过程，代码可以用截图，但每个核心代码都必须加以文字说明，连续的代码不能超过 1 页！下同。

功能描述：用户可以通过图形界面添加新的记账事务，包括选择类型（收入或支出）、输入金额、描述和日期。

实现过程：

1. **前端界面设计：**
   - 在 TransactionList.vue 组件中，添加一个“添加事务”按钮。点击该按钮后，弹出 TransactionForm 对话框。
   - 在 TransactionForm.vue 中，用户填写事务的详细信息，并提交表单。

```
<!-- TransactionList.vue -->
<el-button type="primary" @click="openAddForm">添加事务</el-button>

<!-- TransactionForm.vue -->
<el-button size="small" type="primary" @click="submitForm">确定</el-
button>
```

后端控制器处理：

- 前端提交的事务数据通过 POST /api/transactions 接口发送到后端。
- TransactionController 接收请求，调用 TransactionService 添加事务。

```
// TransactionController.java
@PostMapping
public Transaction addTransaction(@RequestBody Transaction
transaction) {
    return transactionService.addTransaction(transaction);
}
```

服务层逻辑：

TransactionServiceImpl 处理事务的添加逻辑，并通知观察者（如日志记录器）

```
// TransactionServiceImpl.java
@Override
public Transaction addTransaction(Transaction transaction) {
```

```
    Transaction savedTransaction =
transactionRepository.save(transaction);
    // 通知监听器
    for (TransactionListener listener : listeners) {
        listener.onTransactionAdded(savedTransaction);
    }
    return savedTransaction;
}
```

3.5 功能模块 2 删除事务的开发与实现

功能描述：用户可以删除不需要的记账事务

前端界面设计：

- 在事务列表的每一行添加“删除”按钮。点击后弹出确认对话框。

```
<!-- TransactionList.vue -->
<el-button size="small" type="danger"
@click="deleteTransaction(scope.row.id)">删除</el-button>
```

后端控制器处理：

- 前端发送 DELETE /api/transactions/{id} 请求到后端。
- TransactionController 调用 TransactionService 删除指定事务。

```
// TransactionController.java
@DeleteMapping("/{id}")
public String deleteTransaction(@PathVariable Long id) {
    transactionService.deleteTransaction(id);
    return "Transaction deleted successfully.";
}
```

服务层逻辑：

- TransactionServiceImpl 删除事务并通知观察者。

```
// TransactionServiceImpl.java
@Override
public void deleteTransaction(Long id) {
```

```
    Optional<Transaction> optional =
transactionRepository.findById(id);
    if (optional.isPresent()) {
        Transaction transaction = optional.get();
        transactionRepository.deleteById(id);
        // 通知监听器
        for (TransactionListener listener : listeners) {
            listener.onTransactionDeleted(id, transaction);
        }
    } else {
        throw new RuntimeException("Transaction not found with id "
+ id);
    }
}
```

3.6 功能模块 3 修改事务的开发与实现

功能描述：用户可以编辑已有的记账事务，修改类型、金额、描述或日期。

实现过程：

1. **前端界面设计**：
    - 在事务列表的每一行添加“编辑”按钮。点击后弹出 TransactionForm 对话框，表单预填充现有事务数据。

```
<!-- TransactionList.vue -->
<el-button size="small" @click="openEditForm(scope.row)">编辑</el-
button>
```

后端控制器处理：

- 前端提交的修改数据通过 PUT /api/transactions/{id} 接口发送到后端。
- TransactionController 调用 TransactionService 更新事务。

```
// TransactionController.java
@PutMapping("/{id}")
public Transaction updateTransaction(@PathVariable Long id,
@RequestBody Transaction transaction) {
    return transactionService.updateTransaction(id, transaction);
}
```

服务层逻辑：

- TransactionServiceImpl 更新事务并通知观察者。

```
// TransactionServiceImpl.java
@Override
public Transaction updateTransaction(Long id, Transaction
transaction) {
    Optional<Transaction> optional =
transactionRepository.findById(id);
    if (optional.isPresent()) {
        Transaction existing = optional.get();
        existing.setType(transaction.getType());
        existing.setAmount(transaction.getAmount());
        existing.setDescription(transaction.getDescription());
        existing.setDate(transaction.getDate());
        Transaction updated = transactionRepository.save(existing);
        // 通知监听器
        for (TransactionListener listener : listeners) {
            listener.onTransactionUpdated(updated);
        }
        return updated;
    } else {
        throw new RuntimeException("Transaction not found with id "
+ id);
    }
}
```

3.7 功能模块 4 查询事务的开发与实现

功能描述：用户可以查看所有记账事务，并支持按日期、类型等条件进行筛选。

实现过程：

1. **前端界面设计：**
    - 在 TransactionList.vue 中，添加筛选选项，如日期选择器和类型下拉菜单。
    - 用户选择筛选条件后，调用 fetchTransactions 方法，传递筛选参数。

```
<!-- TransactionList.vue -->
<el-select v-model="filterType" placeholder="选择类型"
@change="fetchTransactions">
  <el-option label="全部" value=""></el-option>
  <el-option label="收入" value="INCOME"></el-option>
  <el-option label="支出" value="EXPENSE"></el-option>
</el-select>

<el-date-picker v-model="filterDate" type="date" placeholder="选择日
期" @change="fetchTransactions"></el-date-picker>
```

后端控制器处理：

- 前端发送带有筛选参数的 GET /api/transactions 请求。
- TransactionController 调用 TransactionService 获取符合条件的事务。

```
// TransactionController.java
@GetMapping
public List<Transaction> getAllTransactions(
    @RequestParam(required = false) String type,
    @RequestParam(required = false) @DateTimeFormat(pattern =
"yyyy-MM-dd") Date date) {
    return transactionService.getAllTransactions(type, date);
}
```

服务层逻辑：

- TransactionServiceImpl 根据筛选条件查询事务，并返回结果。

```
// TransactionService.java
List<Transaction> getAllTransactions(String type, Date date);

// TransactionServiceImpl.java
@Override
public List<Transaction> getAllTransactions(String type, Date date)
{
    if (type != null && date != null) {
        return transactionRepository.findByTypeAndDate(type, date);
    } else if (type != null) {
        return transactionRepository.findByType(type);
    } else if (date != null) {
```

```
        return transactionRepository.findByDate(date);
    } else {
        return transactionRepository.findAll();
    }
}
```

3.8 功能模块 5 数据持久化的开发与实现

功能描述：所有记账数据保存到数据库中，确保程序重新启动后能够加载历史数据。

实现过程：

1. **数据库连接配置：**
    - 使用单例模式的 DatabaseConnection 类管理数据库连接，确保全局只有一个连接实例。
    - 配置全局 CORS 设置，允许前端应用访问后端 API。

```
// WebConfig.java
@Configuration
public class WebConfig {

    @Bean
    public WebMvcConfigurer corsConfigurer() {
        return new WebMvcConfigurer() {
            @Override
            public void addCorsMappings(CorsRegistry registry) {
                // 允许所有路径的跨域请求
                registry.addMapping("/**")
                        .allowedOrigins("http://localhost:5173") //
允许的前端地址
                        .allowedMethods("GET", "POST", "PUT",
"DELETE", "OPTIONS") // 允许的 HTTP 方法
                        .allowedHeaders("*") // 允许的请求头
                        .allowCredentials(true); // 是否允许发送凭据
（如 Cookies）
            }
        };
    }
}
```

实体类设计：

- Transaction 类映射数据库中的 transactions 表，包含事务的详细信息。

```
// Transaction.java
@Entity
@Table(name = "transactions")
public class Transaction {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String type; // INCOME 或 EXPENSE

    private Double amount;

    private String description;

    @Temporal(TemporalType.TIMESTAMP)
    private Date date;

    // 构造方法、Getter 和 Setter
}
```

数据操作：

- 通过 TransactionRepository 继承 JpaRepository，实现对事务数据的增删改查操作。
- 在服务层 TransactionServiceImpl 中，调用 TransactionRepository 进行数据库操作，并通过观察者模式通知相关监听器。

```
// TransactionRepository.java
@Repository
public interface TransactionRepository extends
JpaRepository<Transaction, Long> {
    List<Transaction> findByType(String type);
    List<Transaction> findByDate(Date date);
    List<Transaction> findByTypeAndDate(String type, Date date);
}
```

```java
// TransactionServiceImpl.java
@Override
public List<Transaction> getAllTransactions(String type, Date date)
{
    if (type != null && date != null) {
        return transactionRepository.findByTypeAndDate(type, date);
    } else if (type != null) {
        return transactionRepository.findByType(type);
    } else if (date != null) {
        return transactionRepository.findByDate(date);
    } else {
        return transactionRepository.findAll();
    }
}
```

3.9 功能模块 6 图形用户界面的开发与实现

功能描述：提供友好的图形界面，方便用户进行记账操作和查看数据。

实现过程：

1. **界面布局**：
    - 使用 Element Plus 组件库设计直观的用户界面，包括按钮、表格、对话框等。
    - 在 main.js 中引入并使用 Element Plus，确保所有组件在项目中可用。

```javascript
// main.js
import { createApp } from 'vue'
import App from './App.vue'
import ElementPlus from 'element-plus'
import 'element-plus/dist/index.css'

const app = createApp(App)

app.use(ElementPlus)

app.mount('#app')
```

用户交互：

- 通过按钮点击事件、表单提交等交互方式，实现用户与系统的有效沟通。
- 使用 Element Plus 提供的 el-button、el-table、el-dialog 等组件，构建功能丰富的用户界面。

```
<!-- src/components/TransactionList.vue -->
<template>
  <div>
    <el-button type="primary" @click="openAddForm">添加事务</el-button>
    <el-table :data="transactions" style="width: 100%" stripe>
      <el-table-column prop="id" label="ID" width="50"></el-table-
column>
      <el-table-column prop="type" label="类型"></el-table-column>
      <el-table-column prop="amount" label="金额"></el-table-column>
      <el-table-column prop="description" label="描述"></el-table-
column>
      <el-table-column prop="date" label="日期" width="180">
        <template #default="scope">
          {{ formatDate(scope.row.date) }}
        </template>
      </el-table-column>
      <el-table-column label="操作" width="180">
        <template #default="scope">
          <el-button size="small" @click="openEditForm(scope.row)">编辑
</el-button>
          <el-button size="small" type="danger"
@click="deleteTransaction(scope.row.id)">删除</el-button>
        </template>
      </el-table-column>
    </el-table>
```

响应式设计：

- 确保界面在不同设备和屏幕尺寸下均能良好显示和操作。
- 使用 Element Plus 的响应式布局系统，自动调整组件的布局和尺寸。

```
<!-- App.vue -->
<template>
  <div id="app">
    <TransactionList />
  </div>
</template>
```

```
<script>
import TransactionList from './components/TransactionList.vue'

export default {
  name: 'App',
  components: {
    TransactionList
  }
}
</script>

<style>
/* 全局样式 */
#app {
  padding: 20px;
}
</style>
```

4 运行结果测试

以截图的方式展示项目的运行结果，并附加测试过程描述。

本节将通过截图展示电子记账系统的运行结果，并附加详细的测试过程描述，确保各项功能均按预期工作。

4.1 添加事务



测试过程：

1. 点击“添加事务”按钮，弹出添加事务对话框。
2. 选择事务类型为“收入”，输入金额“1500”，描述“Freelance Work”，选择日期“2024-10-24”。
3. 点击“确定”按钮，事务成功添加并显示在事务列表中。

结果：事务成功添加，数据正确显示。

4.2 编辑事务

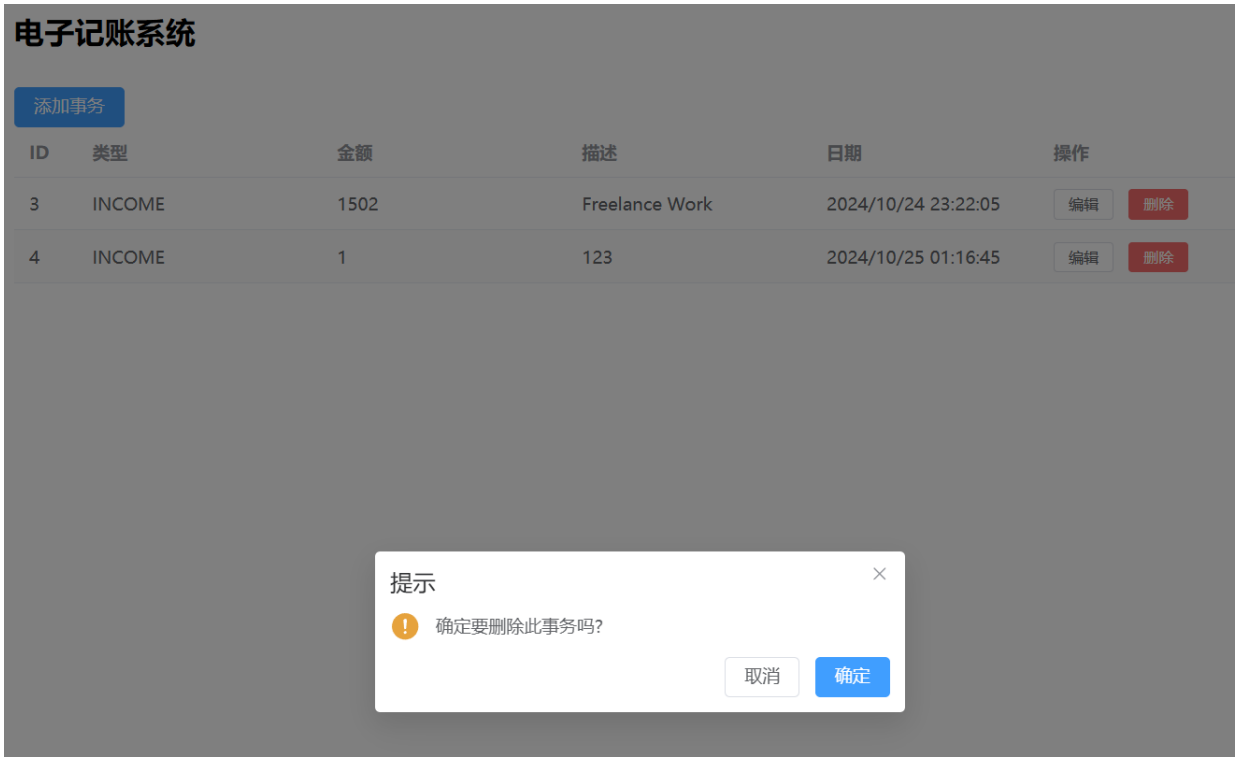


测试过程：

1. 在事务列表中选择一条已有事务，点击“编辑”按钮，弹出编辑事务对话框。
2. 修改金额为“2000”，点击“确定”按钮，事务信息成功更新并显示在列表中。

结果：事务成功更新，数据正确显示。

4.3 删除事务





测试过程：

1. 在事务列表中选择一条事务，点击“删除”按钮，弹出删除确认对话框。
2. 确认删除，事务被成功移除，事务列表更新。

结果：事务成功删除，列表正确更新。

4.4 查询事务

测试过程：

1. 在筛选选项中选择事务类型“收入”，点击“查询”按钮。
2. 系统显示所有类型为“收入”的事务记录。

结果：查询功能正常，符合条件的事务正确显示。

4.5 数据持久化



测试过程：

1. 添加多条事务记录后，关闭程序。
2. 重新启动程序，系统自动加载并显示之前添加的所有事务记录。

结果：数据持久化功能正常，事务记录在程序重启后依然存在。

4.6 图形用户界面



测试过程：

1. 浏览主界面，检查各项功能按钮是否可用。
2. 通过界面进行添加、编辑、删除、查询等操作，确保界面响应正常，操作顺畅。

结果：图形用户界面友好，各项功能通过界面正常操作，用户体验良好。

5 使用设计模式总结

总结本系统使用到的设计模式，并阐述它们在系统中起到的作用

在本电子记账系统中，采用了多种软件设计模式，包括单例模式、工厂模式和观察者模式。这些设计模式在系统中起到了关键作用，提升了系统的可维护性、扩展性和代码质量。

5.1 单例模式

应用场景：数据库连接管理。

实现方式：DatabaseConnection 类采用单例模式，确保整个应用程序中只有一个数据库连接实例。

作用：

- **统一管理数据库连接**：通过单例模式，所有需要数据库连接的模块都使用同一个实例，确保连接的统一性和一致性。
- **节省资源**：避免创建多个数据库连接实例，减少系统资源的消耗。
- **保证数据一致性**：确保所有模块访问同一数据库连接，避免数据不一致的问题。

```
// DatabaseConnection.java
public class DatabaseConnection {
    private static DatabaseConnection instance;
    private Connection connection;

    private DatabaseConnection() {
        // 初始化数据库连接
    }

    public static DatabaseConnection getInstance() {
        if (instance == null) {
            synchronized(DatabaseConnection.class) {
                if(instance == null){
                    instance = new DatabaseConnection();
                }
            }
        }
```

```
        return instance;
    }

    public Connection getConnection() {
        return connection;
    }

    public void disconnect() {
        // 断开数据库连接
    }
}
```

5.2 工厂模式（Factory Pattern）

应用场景：记账事务对象的创建。

实现方式：TransactionFactory 类采用工厂模式，根据不同的事务类型（收入或支出）创建相应的 Transaction 对象。

作用：

- **简化对象创建**：通过工厂方法，隐藏了对象创建的具体实现，调用者无需关心具体的创建细节。
- **提高代码可扩展性**：在需要支持更多事务类型时，只需扩展工厂类，无需修改现有代码。
- **增强代码复用性**：集中管理对象创建逻辑，减少代码重复，提高代码复用性。

```
// TransactionFactory.java
public class TransactionFactory {
    public Transaction createTransaction(String type, Double
amount, String description, Date date) {
        if ("INCOME".equalsIgnoreCase(type)) {
            return new IncomeTransaction(amount, description, date);
        } else if ("EXPENSE".equalsIgnoreCase(type)) {
            return new ExpenseTransaction(amount, description,
date);
        } else {
            throw new IllegalArgumentException("Invalid transaction
type");
        }
```

```
    }
}
```

5.3 观察者模式（Observer Pattern）

应用场景：事务变化的通知机制。

实现方式：TransactionManager 类作为主题（Subject），TransactionListenerImpl 类作为观察者（Observer），在事务发生变化时通知所有注册的观察者。

作用：

- **解耦**：主题与观察者之间通过接口通信，减少了模块之间的直接依赖，提高系统的灵活性。
- **扩展性强**：可以动态添加或移除观察者，满足不同模块对事务变化的不同需求。
- **增强系统功能**：通过观察者模式，可以在事务变化时执行多种操作，如日志记录、实时更新统计数据等。

```
// TransactionListener.java
public interface TransactionListener {
    void onTransactionAdded(Transaction transaction);
    void onTransactionUpdated(Transaction transaction);
    void onTransactionDeleted(Long id, Transaction transaction);
}

// TransactionListenerImpl.java
@Component
public class TransactionListenerImpl implements TransactionListener
{

    @Override
    public void onTransactionAdded(Transaction transaction) {
        System.out.println("事务添加: " + transaction.getId() + ", 类
型: " + transaction.getType() + ", 金额: " +
transaction.getAmount());
    }

    @Override
    public void onTransactionUpdated(Transaction transaction) {
```

```
        System.out.println("事务更新: " + transaction.getId() + ", 类
型: " + transaction.getType() + ", 金额: " +
transaction.getAmount());
    }

    @Override
    public void onTransactionDeleted(Long id, Transaction
transaction) {
        System.out.println("事务删除: " + id + ", 类型: " +
transaction.getType() + ", 金额: " + transaction.getAmount());
    }
}
```

注意事项：

1. 请按照本文档的格式编写
2. 实验完成后将本文档转为 PDF 格式，以“学号-姓名-期末考查.pdf”命名
3. 将工程中的 src 目录打包成 zip 格式，以“学号-姓名-期末考查.zip”命名
4. 将上述 2 个文件发给班级的学习委员，学习委员收集后发给老师（两个文件分开收取，放于 2 个不同的文件夹中）